WEEK 4: Reinforcement Learning for Autonomous Systems

Exploring PPO in Reinforcement Learning for High-Speed Racing Systems

Reinforcement Learning for Autonomous Racing using PPO

Objective

The objective of this project was to develop an autonomous racing agent capable of navigating a 2D racing track using Reinforcement Learning (RL). Unlike supervised learning, where models learn from labeled examples, reinforcement learning allows an agent to learn through trial and error by interacting with an environment and receiving rewards based on its actions.

The goal was to understand how different design choices such as observations, actions, reward functions, and training parameters affect the behavior of an autonomous agent.

Framework Used

- **Gymnasium (CarRacing-v3)**
- **Stable-Baselines3**
- **PPO (Proximal Policy Optimization)**
- **PyTorch (CUDA acceleration)**
- **TensorBoard**

Why PPO?

PPO was selected because:

- It is one of the most widely used on-policy reinforcement learning algorithms.
- It is stable compared to many other policy-gradient algorithms.
- It supports continuous action spaces required by CarRacing.
- Stable-Baselines3 provides a reliable implementation with GPU support.

Algorithms such as DQN were not used because DQN is designed for discrete action spaces, whereas CarRacing requires continuous steering, throttle, and braking. SAC and DDPG were considered but not chosen because the objective was understanding reinforcement learning rather than maximizing performance with more complex algorithms.

Environment

Environment used:

- **Gymnasium CarRacing-v3**

The environment generates a random racing track each episode. The agent starts with no prior knowledge and must learn how to drive using rewards received from the environment.

State (Observation Space)

The agent receives a 96 × 96 RGB image from the environment. Later experiments also used frame stacking, allowing the agent to observe multiple consecutive frames to better estimate motion.

Observation Space:

- Image size: 96 × 96 pixels
- RGB color channels
- Continuous visual observation

The observation allows the agent to estimate:

- Position on the road
- Road direction
- Grass and boundaries
- Upcoming turns

Action Space

The action space is continuous and contains three values:

1. Steering (-1 to +1)
2. Accelerator (0 to 1)
3. Brake (0 to 1)

This allows smooth vehicle control instead of selecting from fixed actions.

Reward Function

Initially, the environment reward was modified by introducing reward shaping.

Experiments included:

- Steering penalties
- Brake penalties

- Throttle penalties
- Preventing simultaneous braking and acceleration

The purpose of reward shaping was to encourage smoother driving behavior. After multiple experiments, the project also tested the default CarRacing reward function because excessive reward shaping did not significantly improve behavior.

The built-in reward already encourages:

- Visiting new road tiles
- Completing the lap
- Avoiding unnecessary time

Training Process

Training Algorithm:

- PPO
- CNN Policy
- GPU acceleration (CUDA)

Hyperparameters used:

- Learning Rate: 3×10^{-4}
- Batch Size: 256
- n_steps: 2048
- Gamma: 0.99
- GAE Lambda: 0.95
- Entropy Coefficient: 0.001
- 4 Parallel Environments

Training was monitored using TensorBoard. Multiple training sessions were performed including:

- 10,000 timesteps
- 100,000 timesteps
- 200,000 timesteps
- 500,000 timesteps

Learning Process

Initially, the agent behaved randomly, often driving onto grass, spinning continuously, colliding with boundaries, and steering randomly. As training progressed, TensorBoard showed a significant improvement in average episode reward, increasing from approximately -70 during early training to over 400 during the best stages of training. This demonstrated that the policy was learning meaningful behaviors from the reward signal.

Failed Behavior

Despite improvements, the final driving behavior was still unsatisfactory. Failures included excessive acceleration, entering corners too quickly, failing to brake sufficiently, spinning after leaving the track, and poor recovery once off-road. Possible reasons include limited observation from image inputs, difficulty estimating vehicle speed, PPO converging to a locally optimal but unstable policy, and reward improvements not perfectly matching driving quality. This demonstrated that higher reward values do not always translate into better real-world behavior.

Improvements Attempted

Experiment 1

Default PPO training.

- *Result:* The vehicle mostly drove randomly and frequently spun.

Experiment 2

Reward shaping.

- *Changes:* Steering penalties, brake penalties, throttle penalties
- *Result:* Training reward improved, but the vehicle continued to over-accelerate and frequently left the track.

Experiment 3

Hyperparameter tuning.

- *Changes:* Increased n_steps to 2048, reduced entropy coefficient to 0.001, parallel environments, GPU training
- *Result:* Training became more stable and faster.

Experiment 4

Frame Stacking.

Four consecutive frames were stacked together to provide temporal information.

- *Purpose:* Better estimation of motion and understanding of turning behavior
- *Result:* Although training stability improved, the final behavior still struggled with aggressive cornering.

Exploration vs Exploitation

During early training, the agent explored many random actions. As training progressed, PPO gradually shifted towards exploitation by repeating actions that produced higher rewards. Reducing the entropy coefficient encouraged the agent to rely more on successful behaviors rather than continuing random exploration.

Why Certain Methods Were Not Used

DQN

Not used because CarRacing requires continuous actions whereas DQN supports discrete actions.

SAC

Although SAC often performs well on continuous control problems, PPO was selected because it is simpler, more widely taught, and easier to analyze for educational purposes.

DDPG

DDPG is sensitive to hyperparameters and often less stable than PPO.

Rule-based Driving

A manually programmed controller was intentionally avoided because the objective was to allow the agent to learn autonomously through interaction with the environment.

Challenges Faced

- Long training times
- Hyperparameter tuning
- Reward design
- Balancing exploration and exploitation
- Evaluating whether higher reward corresponded to better driving behavior
- GPU configuration for PyTorch
- TensorBoard monitoring

Conclusion

This project demonstrated the complete reinforcement learning workflow for autonomous driving. The agent successfully learned from interaction with the environment rather than labeled data. Multiple reward functions and training configurations were evaluated to understand how different design choices influence behavior.

Although the final agent was still unable to complete the track consistently and frequently entered corners with excessive speed, the experiments clearly showed how reinforcement

learning policies evolve through experience and how reward design, observations, and hyperparameters significantly influence learning outcomes.

The project provided practical experience with PPO, Gymnasium, Stable-Baselines3, TensorBoard, and GPU-accelerated reinforcement learning while highlighting the iterative nature of designing and improving autonomous agents.

GitHub link:

GitHub - samarth3134/RL-Autonomous-Racing-Agent: week 4, rl autonomous racing agen...